

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа

Выполнил:

Андронов Денис

Группа

К3342

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

ДЗ6: Настройка Gitlab CI/Github actions для автоматического развёртывания бэкенд-приложения

Срок: 20.05.2026

Задание:

Необходимо настроить автодеплой (с триггером на обновление кода в вашем репозитории, на определённой ветке) для вашего приложения на удалённый сервер с использованием Github Actions или Gitlab CI (любая другая CI-система также может быть использована).

Ход работы

В рамках ЛР4 приложение было развёрнуто на виртуальной машине в Google Cloud (Ubuntu). Использовались Docker Compose и nginx в режиме обратного прокси.

Структура:

- `deploy/` — скрипты настройки сервера, деплоя и nginx;
- `restorator-microservices/` — исходный код микросервисов и `docker-compose.yml`;
- `.github/workflows/hw6-deploy.yml` — workflow GitHub Actions (в корне репозитория).

Параметры сервера:

Параметр	Значение
Платформа	Google Cloud Compute Engine
ОС	Ubuntu
External IP	35.188.15.218
SSH-пользователь	kr4nder
Путь к репозиторию на сервере	/home/kr4nder/ITMO-ACS-Backend-2026-C

Параметр	Значение
Путь к приложению	БР2.1/Андронов Денис/homeworks/hw6

3. Схема автодеплойа

Разработчик → git push (ветка hw6) → GitHub Actions



SSH на сервер



git pull origin hw6



bash deploy/deploy.sh



Docker Compose (микросервисы)



API Gateway (127.0.0.1:4000) ← nginx (:80)

При push в ветку hw6 GitHub Actions подключается к серверу по SSH, обновляет код и запускает скрипт деплоя. Снаружи приложение доступно через nginx, который проксирует запросы на API Gateway.

4. Настройка GitHub Actions

4.1. Workflow

Файл: .github/workflows/hw6-deploy.yml

Триггеры:

- автоматически при push в ветку hw6;
- вручную через Actions → HW6 deploy → Run workflow.

Шаги pipeline:

1. Добавление хоста сервера в known_hosts (ssh-keyscan).
2. Подключение к серверу по SSH (action appleboy/ssh-action@v1.2.5).
3. На сервере:
 - переход в каталог репозитория;
 - git fetch, git checkout hw6, git pull --ff-only origin hw6;
 - переход в homeworks/hw6;
 - выполнение bash deploy/deploy.sh.

Таймаут выполнения SSH-команд — 30 минут (сборка Docker-образов может занимать продолжительное время).

4.2. SSH-доступ для CI

Для GitHub Actions создан отдельная пара SSH-ключей (ed25519):

- публичный ключ добавлен в ~/.ssh/authorized_keys на сервере;
- приватный ключ сохранён в секрете репозитория DEPLOY_SSH_KEY.

Ключ создан без passphrase, чтобы pipeline выполнялся без интерактивного ввода.

4.3. Секреты репозитория

В Settings → Secrets and variables → Actions настроены секреты:

Секрет	Назначение
DEPLOY_HOST	внешний IP сервера (35.188.15.218)

Секрет	Назначение
DEPLOY_USER	имя SSH-пользователя (kr4nder)
DEPLOY_SSH_KEY	приватный SSH-ключ для Actions
DEPLOY_REPO_DIR	абсолютный путь к клону репозитория на сервере
DEPLOY_APP_REL	относительный путь до каталога с deploy/deploy.sh

5. Скрипт деплоя на сервере

Скрипт homeworks/hw6/deploy/deploy.sh выполняет:

1. Остановку старых контейнеров (docker compose down --remove-orphans).
2. Сборку и запуск сервисов:

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
```

3. Ожидание готовности API Gateway — проверка `http://127.0.0.1:4000/health`.

Файл `docker-compose.prod.yml` ограничивает доступ с хоста: API Gateway слушает только `127.0.0.1:4000`, PostgreSQL и RabbitMQ наружу не пробрасываются.

Вывод

В ходе работы настроен автоматический деплой бэкенд-приложения Restorator на удалённый сервер в Google Cloud с использованием GitHub Actions. Pipeline запускается при каждом push в ветку hw6, подключается к серверу по SSH, обновляет код из репозитория и перезапускает приложение через Docker Compose.

Требования задания выполнены:

- использована CI-система GitHub Actions;
- настроен триггер на обновление кода в определённой ветке (hw6);
- развёртывание выполняется на удалённый сервер автоматически;